

Cache Line 微基准测试报告

程序输出的性能数据如下:

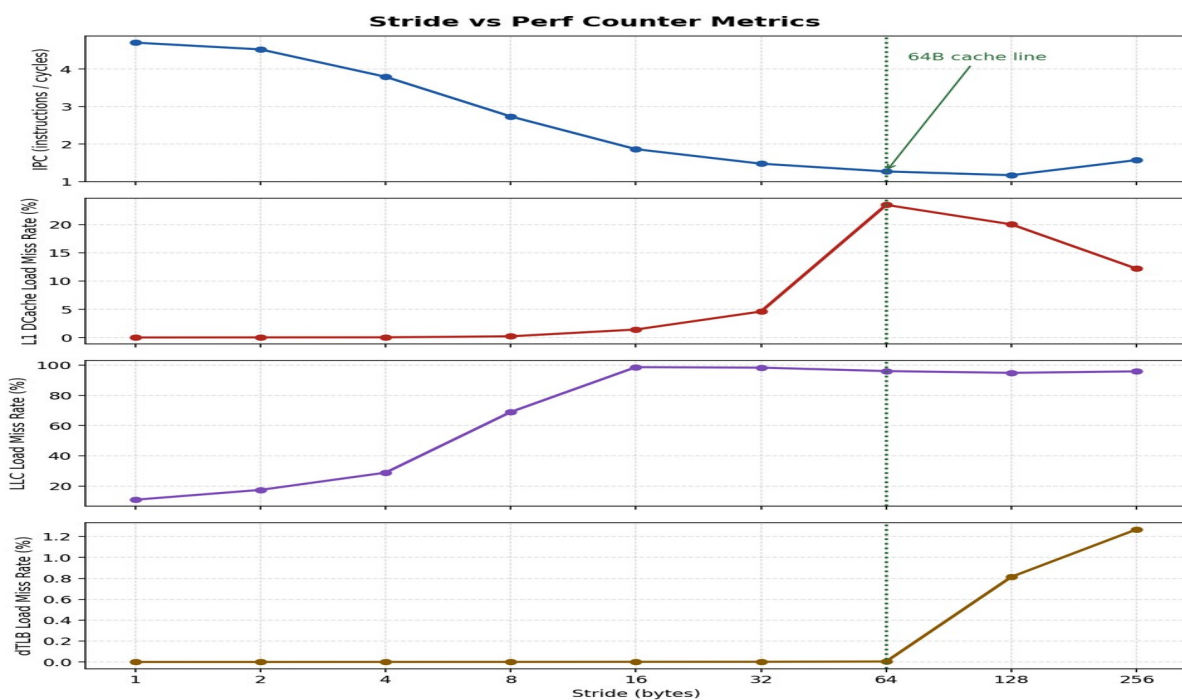
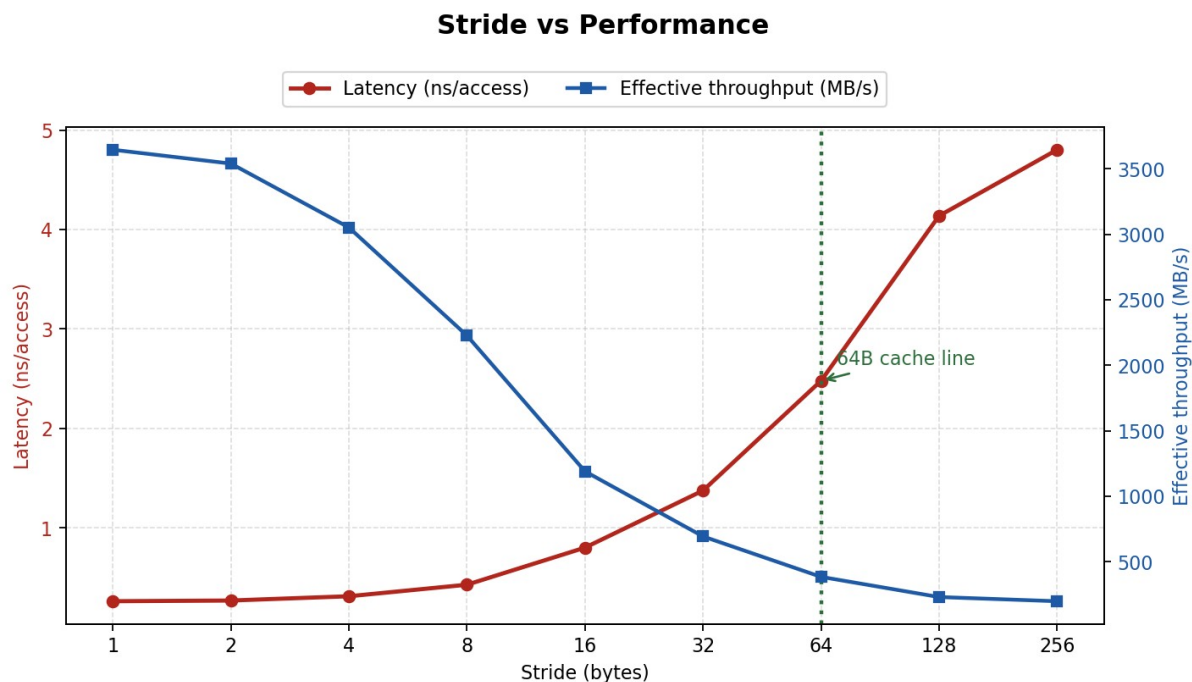
stride (B)	ns/access	effective_MBps
1	0.263	3626.996
2	0.271	3525.493
4	0.318	3001.920
8	0.466	2045.687
16	0.707	1349.034
32	1.439	662.892
64	2.362	403.812
128	3.855	247.388
256	4.502	211.832

perf stat 衍生指标

stride (B)	IPC	L1 Load Miss Rate	L2 Request Miss Rate	LLC Load Miss Rate	dTLB Load Miss Rate
1	4.67	0.03%	65.86%	22.79%	0.00%
2	4.36	0.04%	68.90%	45.21%	0.00%
4	3.55	0.06%	70.07%	57.20%	0.00%
8	2.57	0.28%	75.60%	50.02%	0.00%
16	1.78	1.56%	86.90%	98.56%	0.00%
32	1.34	4.93%	90.35%	97.56%	0.00%
64	1.19	24.84%	82.16%	93.45%	0.01%
128	1.12	20.17%	80.40%	95.12%	0.84%
256	1.30	13.68%	80.32%	103.95%	1.17%

说明:

- `L1 DCache Load Miss Rate = L1-dcache-load-misses / L1-dcache-loads`
- `L2 Request Miss Rate = l2\_rqsts.miss / l2\_rqsts.references`
- `LLC Load Miss Rate = LLC-load-misses / LLC-loads`
- `dTLB Load Miss Rate = dTLB-load-misses / dTLB-loads`



由曲线图可以看出，stride = 64 是拐点。本机的 cache line 均为 64，本实验程序每次显式读取一个 `uint64_t`，即 8B。当访问步长逐渐增大时，同一条 cache line 内能够被有效复用的数据次数不断减少，最终在 stride = 64B 时基本退化为“每次访问一条新的 cache line”。

在小步长阶段，即 1B 到 8B，程序表现出最好的访问性能。此时 ns/access 维持在约 0.263 到 0.466 之间，吞吐量保持在 2000 MB/s 以上。原因是相邻访问高度连续，能够充分复用同一条 cache line。以 stride = 8B 为例，一条 64B cache line 内可以容纳 8 次

uint64_t。读取：64/8=8。

因此，一旦第一次访问将 cache line 拉入缓存，后续多个访问大概率可以直接在 L1 或较近缓存层级中完成。这一阶段在 perf stat 中表现为 IPC 较高，L1 DCache miss rate 极低，dTLB miss rate 基本为 0，说明 CPU 大多数时候都能及时获得数据，后端流水线等待较少。

之后随着

当 stride 增大到 16B 和 32B 后，性能进入中间过渡阶段。此时单条 cache line 内可被利用的访问次数明显下降：

stride = 16B：一条 64B cache line 内约访问 4 次

stride = 32B：一条 64B cache line 内约访问 2 次

cache line 的有效利用率分别下降到约 50% 和 25%。对应地，ns/access 上升，IPC 下降，L1 DCache miss rate 也从不到 1% 上升到 1.56% 和 4.93%。这说明 CPU 已经开始更频繁地等待访存结果，后端 load/store 单元逐渐成为限制因素。

同时，L2 Request Miss Rate 从 75.60% 上升到 86.90%、90.35%，说明越来越多 L2 请求无法在 L2 中命中，需要继续访问更低层级的缓存或内存。因此，L2 指标补充说明了 L1 miss 之后的进一步传播路径：随着 stride 增大，访存请求不仅更容易错过 L1，也更容易穿透到更深的 cache 层级。

stride = 64B 是本实验最关键的转折点。此时步长正好等于典型 cache line 大小，访问模式变成：

第 1 次访问：cache line 0 中的 8B

第 2 次访问：cache line 1 中的 8B

第 3 次访问：cache line 2 中的 8B

也就是说，每次访问基本都会落到一条新的 cache line。程序每次只使用其中 8B 数据，但硬件仍然需要加载整条 64B cache line，因此 cache line 的有效利用率下降到：

$$8B / 64B = 12.5\%$$

这意味着 line 内空间局部性几乎完全丢失。ns/access 继续上升，effective_MBps 和 IPC 下降，L1 DCache miss rate 从 4.93% 大幅上升到 24.84%。这些指标共同说明，CPU 在 stride = 64B 后需要更频繁地为新的 cache line 支付访问代价，访存等待显著增加。

这里最有说服力的证据不是单独某一个指标，而是多个指标的一致变化：延迟升高、吞吐下降、IPC 降低、L1 miss rate 急剧上升。这说明性能下降不是测量噪声，而是由 cache line 复用率断崖式下降导致的微架构瓶颈。

虽然 L2 Request Miss Rate 在 64B 处没有继续上升，反而从 90.35% 回落到 82.16%。这并不表示性能变好，因为 L2 Request Miss Rate 不是判断 64B cache line 边界的最直接指标。一方面，stride = 64B 时 line 内复用断崖已经发生，主要问题首先体现在 L1 DCache miss rate、IPC

和访问延迟上；另一方面，L2/LLC 事件会受到硬件预取器、访问次数变化、事件定义口径等因素影响。因此，L2 Request Miss Rate 更适合作为辅助证据，而不是 cache line 边界拐点的主要判断依据。

当 stride 继续扩大到 128B 和 256B 后，性能继续下降，但没有再次出现像 32B -> 64B 那样的本质转折。原因是从 64B 开始，单条 cache line 的复用已经基本丢失；继续增大步长，只是让访问地址更加稀疏。例如：

stride = 128B: 每次访问跨过 2 条 cache line 的距离
stride = 256B: 每次访问跨过 4 条 cache line 的距离

此时 ns/access 继续上升，IPC 维持在低位，说明 CPU 仍然受到较强的访存延迟限制。更重要的是，dTLB miss rate 开始上升，说明大步长访问已经不只是 cache line 利用率问题，还开始破坏页级局部性。虽然 L1 DCache Load Miss Rate 在 stride=64B 达到明显高点后，在 128B 和 256B 处有所回落。这并不说明大 stride 的访存更快，可能是因为固定步长访问可能被硬件预取器识别，部分后续 cache line 被提前加载。此外，L1-dcache-loads 是 PMU 事件口径下的 load 计数，不完全等同于程序访问次数，因此 miss rate 会受到事件定义、预取器和采样波动影响。结合 ns_per_access 和 IPC 可以看到，128B/256B 的整体性能仍然较差，并没有真正变好。

从页表和 TLB 的角度看，随着 stride 增大，每个页面内的访问次数减少，程序更快跨越大量虚拟页，TLB entry 的复用率下降，因此 dTLB miss rate 在大步长下开始明显升高。128B 和 256B 阶段的性能下降，除了 cache line 复用已经丢失之外，还叠加了 TLB 压力和更深层内存访问开销。

综上，stride = 64B 是本实验的主要 Cache Line 边界拐点。

微架构原因可以概括为：当 stride 小于 64B 时，一个 cache line 可以被多次访问，空间局部性较好；当 stride 达到 64B 时，每次访问基本落到新的 cache line，程序只使用 8B 数据却需要承担整条 64B cache line 的加载成本，导致 L1 DCache miss rate 大幅上升、IPC 下降、访问延迟升高、有效吞吐量下降；当 stride 继续增大到 128B 和 256B 后，性能进一步下降主要来自访问更稀疏、硬件预取效果受限以及 dTLB miss 增加。

火焰图对比

本次重点分析

- `flamegraphs/stride\_1\_flame.svg`

- `flamegraphs/stride\_64\_flame.svg`

5.1 `stride=1`

stride=1 时，相邻访问非常密集。由于程序每次显式读取一个 uint64_t，即 8B，而相邻访问只移动 1B，因此多个访问窗口高度重叠，并且大量访问落在同一条 64B cache line 内。一次 cache line 被加载到缓存后，后续多个访问大概率可以在 L1 或较近缓存层级命中。因此，该场景具有较强的空间局部性，典型路径更接近 Load -> L1 hit -> 数据返回，Cache Miss 较少，CPU 时间主要集中在用户态循环本身，火焰图表现为 run_stride_read 占据绝

大部分宽度。

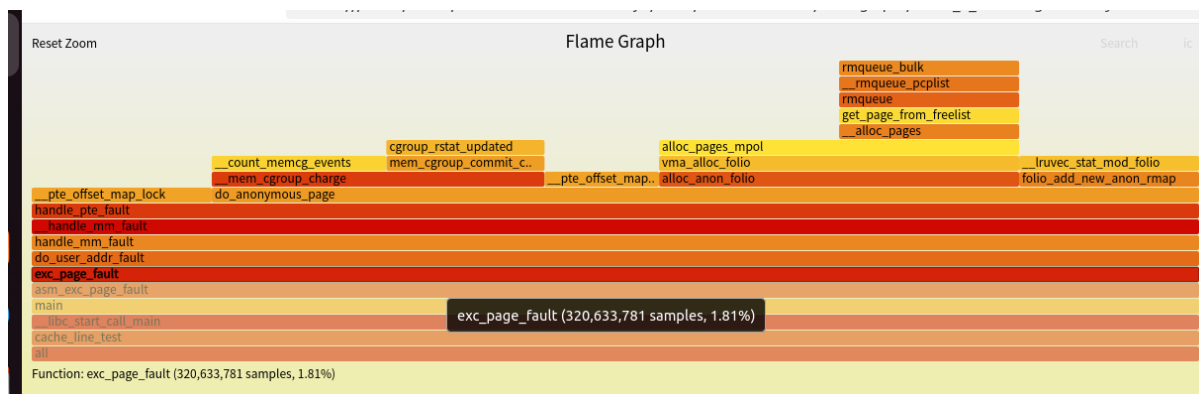
图中虽然也能看到：

handle_mm_fault

do_anonymous_page

alloc_anon_folio

但占比都不高（见下图），更多对应初始化和首次触页阶段，而不是正式计时阶段的主要瓶颈。



5.2 `stride=64`

`stride=64` 的火焰图仍以：

main -> run_stride_read

为主，但 `run\_stride\_read` 占比下降到约 `85.57%`，于 64B 正好对应典型 cache line 大小，每次访问基本落到新的 cache line。程序每次只使用 8B 数据，却需要硬件加载整条 64B cache line，cache line 有效利用率仅约 12.5%。因此，更多 load 指令会经历 L1 DCache miss -> L2 查询 -> LLC 查询 -> 必要时访问内存 -> cache line 回填 的长路径。该路径由硬件完成，所以不会在火焰图中展开为单独的软件调用栈，而是表现为 run_stride_read 中包含更多访存等待时间。同时内核态内存管理路径显著变宽，例如：

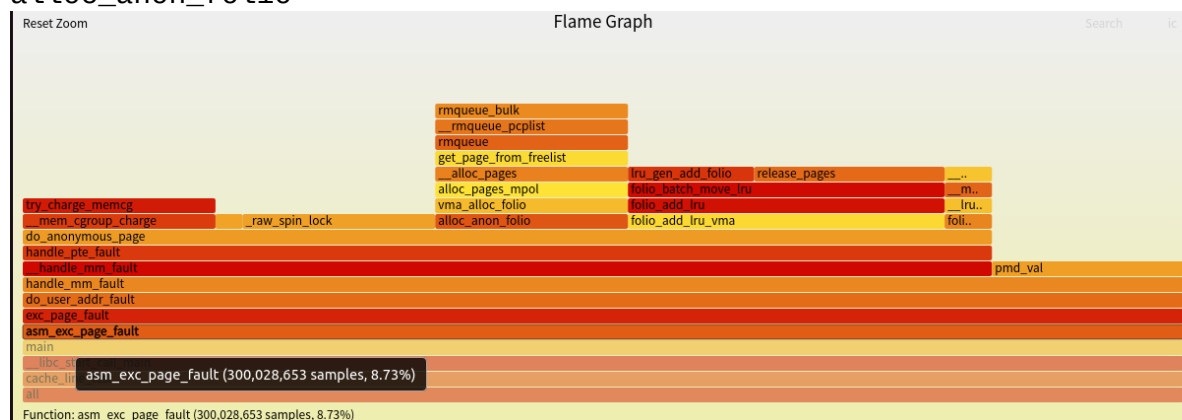
asm_exc_page_fault

do_user_addr_fault

handle_mm_fault

do_anonymous_page

alloc_anon_folio



这些函数说明采样捕获到了匿名页分配、首次触页和内存管理开销。

再结合 run_stride_read 的源码可知，该函数核心操作是按步长读取数组；再结合

stride=64 下 L1 DCache miss rate 升高、IPC 降低、ns/access 上升和吞吐量下降，可以推

断 stride=64 的 run_stride_read 内部发生了更多缓存层级穿透和访存等待。

AI 辅助说明

实际过程中，AI 主要参与了以下工作：

1. 设计微基准程序结构，包括参数格式、warm-up、计时方式和 CSV 输出格式
2. 比较 `uint64\_t` 与 `uint8\_t` 读取方式，最终调整为单字节读取，使 stride 语义更清晰
3. 增加 `64B` 地址对齐和显式对齐校验
4. 分析混合架构 CPU 对 `perf stat` 输出的影响，并建议使用 `taskset` 锁核
5. 编写曲线图脚本，自动生成 stride vs 性能图和 perf 指标图
7. 解释火焰图中的 page fault 路径与普通 cache miss 的区别